Eigenvalue_Linear_Analysis_final

April 14, 2026

1 Linear Algebra Geometric Approach: Eigenvalue Analysis

1.1 Objective

Use **eigenvalue decomposition** to understand the geometric structure of features and their relationship with `perceived_stress_score`.

1.2 Methodology Overview

1.2.1 Core Idea

- Place features in a **coordinate space**
- Decompose the **covariance matrix** to find principal directions
- Analyze which directions are most important for explaining variance
- Examine how these directions relate to the target variable

1.2.2 Mathematical Foundation

Covariance Matrix Eigendecomposition:

$$\text{Cov}(X) = V\Lambda V^T$$

Where: - $\Lambda = \text{diag}(\lambda_1, \lambda_2, ..., \lambda_p)$ are **eigenvalues** - $V = [v_1, v_2, ..., v_p]$ are **eigenvectors** (principal directions) - Eigenvalues represent **variance explained** in each direction

---

1.3 1. Import Libraries

[35]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from scipy import stats
import warnings
```

```
warnings.filterwarnings('ignore')

# Set plot style
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

1.4 2. Define Features and Target

[36]:
```
# Define feature list (20 features)
FEATURES = [

    'daily_active_minutes_instagram',
    'time_on_feed_per_day',
    'time_on_reels_per_day',
    'time_on_explore_per_day',
    'average_session_length_minutes',
    'likes_given_per_day',
    'user_engagement_score',
    'comments_written_per_day',
    'dms_received_per_week',
    'dms_sent_per_week',
    'time_on_messages_per_day',
    'age',
    'exercise_hours_per_week',
    'volunteer_hours_per_month',
    'sleep_hours_per_night'
]

# Target variable
TARGET = 'perceived_stress_score'

print(f"Number of features: {len(FEATURES)}")
print(f"Target variable: {TARGET}")
```

```
Number of features: 15
Target variable: perceived_stress_score
```

1.5 3. Load and Preprocess Data

[37]:
```
# Load data
df = pd.read_csv('instagram_usage_lifestyle_linear2.csv')
print(f"Dataset shape: {df.shape}")
df.head()
```

```
Dataset shape: (1547896, 58)
```

[37]:
```
   user_id   app_name  age      gender         country urban_rural  \
0        1  Instagram   51      Female           India       Rural
1        2  Instagram   64      Female  United Kingdom       Urban
2        3  Instagram   41      Female          Canada       Urban
3        4  Instagram   27  Non-binary     South Korea       Urban
4        5  Instagram   55        Male           India       Urban

   income_level   employment_status education_level relationship_status  …  \
0          High             Retired      Bachelor's              Single  …
1        Middle  Full-time employed           Other            Divorced  …
2        Middle             Student      Bachelor's   In a relationship  …
3        Middle          Unemployed        Master's   In a relationship  …
4  Upper-middle  Full-time employed      Bachelor's              Single  …

  last_login_date  average_session_length_minutes  content_type_preference  \
0      2025-11-02                            18.8                    Mixed
1      2025-03-22                            34.7                   Photos
2      2025-08-10                            38.5                    Mixed
3      2025-03-31                            35.1                  Stories
4      2025-03-19                            10.1                   Videos

  preferred_content_theme privacy_setting_level two_factor_auth_enabled  \
0                    Tech               Private                      Yes
1                 Fashion                Public                       No
2                   Other                Public                      Yes
3                    Tech               Private                       No
4                    Food                Public                      Yes

   biometric_login_used  linked_accounts_count  subscription_status  \
0                    No                      0                 Free
1                    No                      3                 Free
2                   Yes                      1                 Free
3                    No                      1                 Free
4                    No                      0                 Free

   user_engagement_score
0                   6.02
1                   6.32
2                   5.26
```

```
3                    5.41
4                    7.24

[5 rows x 58 columns]
```

[38]:
```
# Handle categorical variable: diet_quality
# Use ordinal encoding (Poor=1, Fair=2, Good=3, Excellent=4)
df_processed = df.copy()

diet_quality_mapping = {
    'Poor': 1,
    'Fair': 2,
    'Average': 2,
    'Good': 3,
    'Excellent': 4
}

df_processed['diet_quality_encoded'] = df_processed['diet_quality'].
 ↪map(diet_quality_mapping)

# If mapping fails, use LabelEncoder
if df_processed['diet_quality_encoded'].isna().sum() > 0:
    le = LabelEncoder()
    df_processed['diet_quality_encoded'] = le.
 ↪fit_transform(df_processed['diet_quality'].astype(str))
    print(f"LabelEncoder mapping: {dict(zip(le.classes_, le.transform(le.
 ↪classes_)))}")
else:
    print("Ordinal encoding applied successfully!")

# Update feature list
FEATURES_ENCODED = [f if f != 'diet_quality' else 'diet_quality_encoded' for f␣
 ↪in FEATURES]
print(f"\nEncoded features: {len(FEATURES_ENCODED)}")
```

```
LabelEncoder mapping: {'Average': np.int64(0), 'Excellent': np.int64(1), 'Good':
np.int64(2), 'Poor': np.int64(3), 'Very poor': np.int64(4)}

Encoded features: 15
```

[39]:
```
# Extract feature matrix X and target vector y
X = df_processed[FEATURES_ENCODED].values.astype(np.float32)
y = df_processed[TARGET].values.astype(np.float32)

# Handle missing values with median
for i in range(X.shape[1]):
    col = X[:, i]
```

```
    mask = np.isnan(col)
    if mask.sum() > 0:
        col[mask] = np.nanmedian(col)

y = np.nan_to_num(y, nan=np.nanmedian(y))

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

```
X shape: (1547896, 15)
y shape: (1547896,)
```

[40]:
```
# Standardize features (crucial for eigenvalue analysis)
# This ensures all features are on the same scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Also standardize y for correlation analysis
y_scaled = (y - y.mean()) / y.std()

print("Standardization completed!")
print(f"X_scaled mean: {X_scaled.mean():.6f}, std: {X_scaled.std():.6f}")
```

```
Standardization completed!
X_scaled mean: 0.000000, std: 1.000000
```

1.6 4. Covariance Matrix Analysis

The covariance matrix captures the **linear relationships** between all pairs of features.

$$\mathrm{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{k=1}^{n} (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)$$

[41]:
```
# Compute covariance matrix
# For standardized data, covariance matrix = correlation matrix
cov_matrix = np.cov(X_scaled.T)

print(f"Covariance matrix shape: {cov_matrix.shape}")
print(f"\nCovariance matrix (first 5x5 block):")
print(np.round(cov_matrix[:5, :5], 3))
```

```
Covariance matrix shape: (15, 15)

Covariance matrix (first 5x5 block):
[[1.    0.542 0.518 0.48  0.409]
 [0.542 1.    0.447 0.405 0.256]
 [0.518 0.447 1.    0.393 0.242]
```

```
 [0.48  0.405 0.393 1.    0.223]
 [0.409 0.256 0.242 0.223 1.   ]]
```

[42]:
```
# # Visualize covariance/correlation matrix
# fig, ax = plt.subplots(figsize=(14, 12))

# # Create heatmap
# sns.heatmap(
#     cov_matrix,
#     annot=False,  # Too many features for annotations
#     cmap='RdBu_r',
#     center=0,
#     xticklabels=FEATURES_ENCODED,
#     yticklabels=FEATURES_ENCODED,
#     ax=ax
# )

# ax.set_title('Feature Covariance Matrix (Standardized Data = Correlation␣
↪Matrix)', fontsize=14)
# plt.xticks(rotation=45, ha='right')
# plt.yticks(rotation=0)
# plt.tight_layout()
# plt.savefig('covariance_matrix.png', dpi=150, bbox_inches='tight')
# plt.show()
```

1.7 5. Eigendecomposition of Covariance Matrix

Decompose the covariance matrix to find: - **Eigenvalues ( )**: Amount of variance in each principal direction - **Eigenvectors (v)**: The principal directions themselves

[43]:
```
# Perform eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Eigenvalues should be real for symmetric matrices
eigenvalues = eigenvalues.real
eigenvectors = eigenvectors.real

# Sort by eigenvalue (descending order)
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print("Eigendecomposition completed!")
print(f"Number of eigenvalues: {len(eigenvalues)}")
```

```
Eigendecomposition completed!
Number of eigenvalues: 15
```

[44]:
```
# Display eigenvalues and variance explained
total_variance = eigenvalues.sum()
variance_ratio = eigenvalues / total_variance
cumulative_variance = np.cumsum(variance_ratio)

print("Eigenvalue Analysis:")
print("=" * 70)
print(f"{'PC':<6} {'Eigenvalue':<15} {'Var. Ratio (%)':<18} {'Cumulative (%)':
↪<15}")
print("-" * 70)

for i in range(len(eigenvalues)):
    print(f"PC{i+1:<4} {eigenvalues[i]:<15.4f} {variance_ratio[i]*100:<18.2f}␣
↪{cumulative_variance[i]*100:<15.2f}")
```

```
Eigenvalue Analysis:
======================================================================
PC     Eigenvalue      Var. Ratio (%)     Cumulative (%)
----------------------------------------------------------------------
PC1    5.6599          37.73              37.73
PC2    1.6330          10.89              48.62
PC3    1.3346          8.90               57.52
PC4    0.9445          6.30               63.81
PC5    0.8308          5.54               69.35
PC6    0.6831          4.55               73.91
PC7    0.6094          4.06               77.97
PC8    0.5912          3.94               81.91
PC9    0.5548          3.70               85.61
PC10   0.5134          3.42               89.03
PC11   0.4329          2.89               91.92
PC12   0.4224          2.82               94.73
PC13   0.3562          2.37               97.11
PC14   0.3441          2.29               99.40
PC15   0.0896          0.60               100.00
```

[45]:
```
# Create eigenvalue summary dataframe
eigenvalue_df = pd.DataFrame({
    'Principal_Component': [f'PC{i+1}' for i in range(len(eigenvalues))],
    'Eigenvalue': eigenvalues,
    'Variance_Ratio': variance_ratio,
    'Cumulative_Variance': cumulative_variance
})

eigenvalue_df
```

[45]:

	Principal_Component	Eigenvalue	Variance_Ratio	Cumulative_Variance
0	PC1	5.659944	0.377329	0.377329
1	PC2	1.633001	0.108867	0.486196

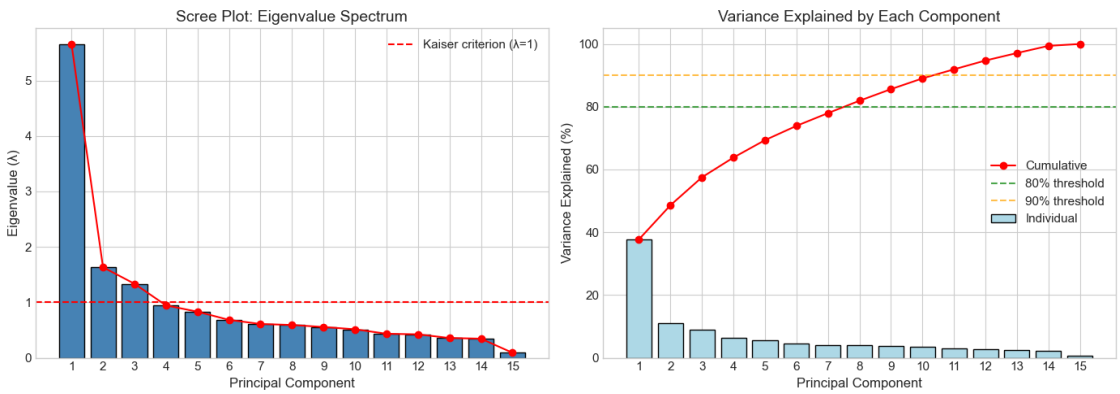
2	PC3	1.334573	0.088971	0.575168
3	PC4	0.944541	0.062969	0.638137
4	PC5	0.830837	0.055389	0.693526
5	PC6	0.683063	0.045537	0.739064
6	PC7	0.609435	0.040629	0.779692
7	PC8	0.591235	0.039416	0.819108
8	PC9	0.554820	0.036988	0.856096
9	PC10	0.513393	0.034226	0.890322
10	PC11	0.432851	0.028857	0.919179
11	PC12	0.422398	0.028160	0.947339
12	PC13	0.356229	0.023749	0.971087
13	PC14	0.344075	0.022938	0.994026
14	PC15	0.089614	0.005974	1.000000

1.8 6. Visualize Eigenvalue Spectrum

The **scree plot** shows how variance is distributed across principal components.

[46]:
```
# Create scree plot and cumulative variance plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Scree Plot (Eigenvalues)
axes[0].bar(range(1, len(eigenvalues)+1), eigenvalues, color='steelblue',␣
↪edgecolor='black')
axes[0].plot(range(1, len(eigenvalues)+1), eigenvalues, 'ro-', markersize=6)
axes[0].set_xlabel('Principal Component', fontsize=12)
axes[0].set_ylabel('Eigenvalue ( )', fontsize=12)
axes[0].set_title('Scree Plot: Eigenvalue Spectrum', fontsize=14)
axes[0].set_xticks(range(1, len(eigenvalues)+1))

# Add horizontal line at  =1 (Kaiser criterion)
axes[0].axhline(y=1, color='red', linestyle='--', label='Kaiser criterion␣
↪( =1)')
axes[0].legend()

# Plot 2: Cumulative Variance Explained
axes[1].bar(range(1, len(eigenvalues)+1), variance_ratio * 100,␣
↪color='lightblue', edgecolor='black', label='Individual')
axes[1].plot(range(1, len(eigenvalues)+1), cumulative_variance * 100, 'ro-',␣
↪markersize=6, label='Cumulative')
axes[1].set_xlabel('Principal Component', fontsize=12)
axes[1].set_ylabel('Variance Explained (%)', fontsize=12)
axes[1].set_title('Variance Explained by Each Component', fontsize=14)
axes[1].set_xticks(range(1, len(eigenvalues)+1))

# Add horizontal lines at common thresholds
```

```
axes[1].axhline(y=80, color='green', linestyle='--', alpha=0.7, label='80%␣
↪threshold')
axes[1].axhline(y=90, color='orange', linestyle='--', alpha=0.7, label='90%␣
↪threshold')
axes[1].legend(loc='center right')

plt.tight_layout()
plt.savefig('eigenvalue_spectrum.png', dpi=150, bbox_inches='tight')
plt.show()

# Report key findings
n_components_80 = np.argmax(cumulative_variance >= 0.80) + 1
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1
n_kaiser = np.sum(eigenvalues >= 1)

print(f"\nKey Findings:")
print(f"  - Components needed for 80% variance: {n_components_80}")
print(f"  - Components needed for 90% variance: {n_components_90}")
print(f"  - Components with    1 (Kaiser criterion): {n_kaiser}")
```




```
Key Findings:
  - Components needed for 80% variance: 8
  - Components needed for 90% variance: 11
  - Components with    1 (Kaiser criterion): 3
```

1.9 7. Eigenvector Analysis (Principal Directions)

Each eigenvector shows **which original features contribute to each principal direction**.

[47]:
```
# Create eigenvector (loadings) dataframe
# Rows = original features, Columns = principal components
```

```python
loadings_df = pd.DataFrame(
    eigenvectors,
    index=FEATURES_ENCODED,
    columns=[f'PC{i+1}' for i in range(len(eigenvalues))]
)

print("Eigenvector Loadings (first 5 PCs):")
print("=" * 70)
print("(Values show how much each feature contributes to each PC)")
print()
loadings_df.iloc[:, :5].round(3)
```

```
Eigenvector Loadings (first 5 PCs):
======================================================================
(Values show how much each feature contributes to each PC)
```

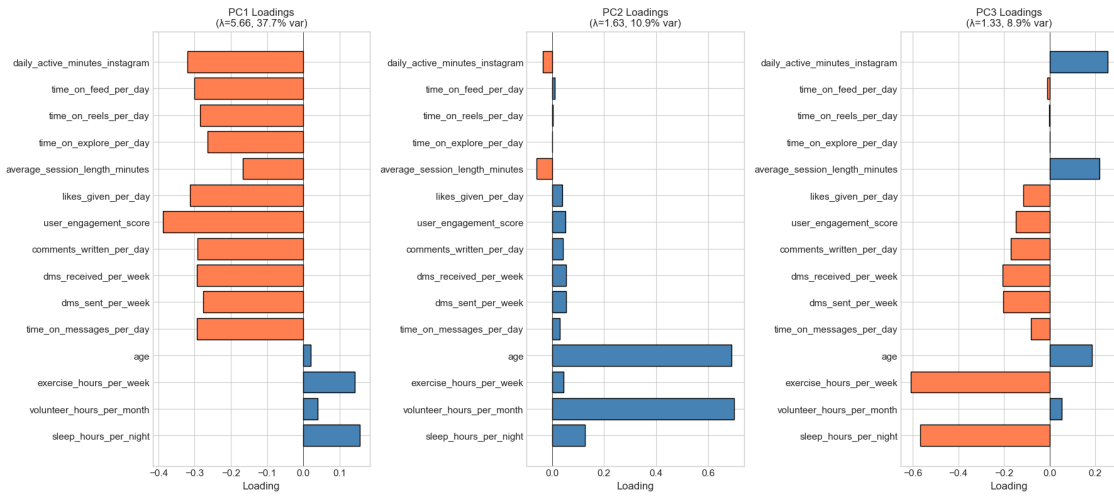
[47]:

	PC1	PC2	PC3	PC4	PC5
daily_active_minutes_instagram	-0.321	-0.034	0.255	0.198	-0.047
time_on_feed_per_day	-0.301	0.010	-0.011	0.165	0.305
time_on_reels_per_day	-0.285	0.004	-0.001	0.148	0.160
time_on_explore_per_day	-0.264	0.002	-0.000	0.119	-0.042
average_session_length_minutes	-0.166	-0.059	0.219	0.730	-0.412
likes_given_per_day	-0.312	0.039	-0.116	0.056	0.416
user_engagement_score	-0.388	0.052	-0.148	-0.092	0.072
comments_written_per_day	-0.292	0.043	-0.170	-0.028	0.345
dms_received_per_week	-0.293	0.055	-0.207	-0.262	-0.317
dms_sent_per_week	-0.276	0.054	-0.203	-0.307	-0.449
time_on_messages_per_day	-0.293	0.031	-0.082	-0.104	-0.325
age	0.021	0.690	0.185	-0.024	0.003
exercise_hours_per_week	0.142	0.044	-0.609	0.313	-0.037
volunteer_hours_per_month	0.040	0.698	0.054	0.077	-0.012
sleep_hours_per_night	0.157	0.127	-0.568	0.272	-0.031

[48]:

```python
# Visualize loadings for first 3 principal components
fig, axes = plt.subplots(1, 3, figsize=(18, 8))

for i, ax in enumerate(axes):
    loadings = loadings_df[f'PC{i+1}'].values
    colors = ['steelblue' if l >= 0 else 'coral' for l in loadings]

    y_pos = np.arange(len(FEATURES_ENCODED))
    ax.barh(y_pos, loadings, color=colors, edgecolor='black')
    ax.set_yticks(y_pos)
    ax.set_yticklabels(FEATURES_ENCODED)
    ax.set_xlabel('Loading', fontsize=12)
    ax.set_title(f'PC{i+1} Loadings\n( ={eigenvalues[i]:.2f},␣
↪{variance_ratio[i]*100:.1f}% var)', fontsize=12)
```

```
    ax.axvline(x=0, color='black', linewidth=0.5)
    ax.invert_yaxis()

plt.tight_layout()
plt.savefig('eigenvector_loadings.png', dpi=150, bbox_inches='tight')
plt.show()
```



[49]:
```
# Identify dominant features for each PC
print("Dominant Features for Each Principal Component:")
print("=" * 70)
print("(Features with |loading| > 0.3)")
print()

for i in range(min(5, len(eigenvalues))):  # First 5 PCs
    pc_name = f'PC{i+1}'
    loadings = loadings_df[pc_name]

    # Find features with high loadings
    high_loadings = loadings[np.abs(loadings) > 0.3].sort_values(key=abs,␣
↪ascending=False)

    print(f"{pc_name} ( ={eigenvalues[i]:.3f}, {variance_ratio[i]*100:.1f}%␣
↪variance):")
    for feat, load in high_loadings.items():
        direction = "+" if load > 0 else "-"
        print(f"    {direction} {feat}: {load:.3f}")
    print()
```

```
Dominant Features for Each Principal Component:
======================================================================
```

```
(Features with |loading| > 0.3)

PC1 ( =5.660, 37.7% variance):
    - user_engagement_score: -0.388
    - daily_active_minutes_instagram: -0.321
    - likes_given_per_day: -0.312
    - time_on_feed_per_day: -0.301

PC2 ( =1.633, 10.9% variance):
    + volunteer_hours_per_month: 0.698
    + age: 0.690

PC3 ( =1.335, 8.9% variance):
    - exercise_hours_per_week: -0.609
    - sleep_hours_per_night: -0.568

PC4 ( =0.945, 6.3% variance):
    + average_session_length_minutes: 0.730
    + exercise_hours_per_week: 0.313
    - dms_sent_per_week: -0.307

PC5 ( =0.831, 5.5% variance):
    - dms_sent_per_week: -0.449
    + likes_given_per_day: 0.416
    - average_session_length_minutes: -0.412
    + comments_written_per_day: 0.345
    - time_on_messages_per_day: -0.325
    - dms_received_per_week: -0.317
    + time_on_feed_per_day: 0.305
```

1.10 8. Project Data onto Principal Component Space

Transform original features into the new coordinate system defined by eigenvectors.

[50]:
```
# Project data onto principal components
# X_pc = X_scaled @ eigenvectors
X_pc = X_scaled @ eigenvectors

print(f"Original data shape: {X_scaled.shape}")
print(f"Projected data shape: {X_pc.shape}")
print("\nData has been transformed to principal component space.")
```

```
Original data shape: (1547896, 15)
Projected data shape: (1547896, 15)

Data has been transformed to principal component space.
```

[51]:
```
# Visualize data in PC1-PC2 space, colored by stress score
fig, ax = plt.subplots(figsize=(10, 8))

# Use a subset for visualization (too many points)
sample_size = min(5000, len(X_pc))
idx_sample = np.random.choice(len(X_pc), sample_size, replace=False)

scatter = ax.scatter(
    X_pc[idx_sample, 0],
    X_pc[idx_sample, 1],
    c=y[idx_sample],
    cmap='RdYlBu_r',
    alpha=0.5,
    s=10
)

ax.set_xlabel(f'PC1 ({variance_ratio[0]*100:.1f}% variance)', fontsize=12)
ax.set_ylabel(f'PC2 ({variance_ratio[1]*100:.1f}% variance)', fontsize=12)
ax.set_title('Data Projected onto PC1-PC2 Space\n(Colored by Perceived Stress␣
↪Score)', fontsize=14)

cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Perceived Stress Score', fontsize=11)

plt.tight_layout()
plt.savefig('pc_projection_2d.png', dpi=150, bbox_inches='tight')
plt.show()
```

1.11 9. Relationship Between Principal Components and Target Variable

Analyze **which principal directions are most correlated with perceived_stress_score**.

[52]:
```
# Calculate correlation between each PC and target variable
pc_target_correlations = []

print("Correlation Between Principal Components and Perceived Stress Score:")
print("=" * 70)
print(f"{'PC':<8} {'Correlation (r)':<18} {'R²':<12} {'p-value':<15}")
print("-" * 70)

for i in range(len(eigenvalues)):
    r, p = stats.pearsonr(X_pc[:, i], y)
    r2 = r ** 2
```

```
    pc_target_correlations.append({'PC': f'PC{i+1}', 'r': r, 'R2': r2,␣
↪'p_value': p})

    # Significance indicator
    sig = '***' if p < 0.001 else ('**' if p < 0.01 else ('*' if p < 0.05 else␣
↪''))
    print(f"PC{i+1:<6} {r:<18.4f} {r2:<12.4f} {p:<12.2e} {sig}")

print("-" * 70)
print("Significance: *** p<0.001, ** p<0.01, * p<0.05")
```

```
Correlation Between Principal Components and Perceived Stress Score:
======================================================================
PC       Correlation (r)    R²           p-value
----------------------------------------------------------------------
PC1      -0.6890            0.4748       0.00e+00     ***
PC2      0.0570             0.0032       0.00e+00     ***
PC3      -0.0998            0.0100       0.00e+00     ***
PC4      -0.0237            0.0006       1.58e-190    ***
PC5      0.0810             0.0066       0.00e+00     ***
PC6      0.0640             0.0041       0.00e+00     ***
PC7      -0.0163            0.0003       1.80e-91     ***
PC8      -0.0023            0.0000       4.83e-03     **
PC9      -0.0207            0.0004       1.13e-146    ***
PC10     0.0094             0.0001       1.63e-31     ***
PC11     0.0083             0.0001       3.15e-25     ***
PC12     0.0171             0.0003       7.01e-100    ***
PC13     -0.0244            0.0006       1.66e-202    ***
PC14     0.0076             0.0001       4.85e-21     ***
PC15     -0.0185            0.0003       7.59e-117    ***
----------------------------------------------------------------------
Significance: *** p<0.001, ** p<0.01, * p<0.05
```

[53]:
```
# Create correlation dataframe
pc_corr_df = pd.DataFrame(pc_target_correlations)
pc_corr_df['Abs_r'] = np.abs(pc_corr_df['r'])
pc_corr_df_sorted = pc_corr_df.sort_values('Abs_r', ascending=False)

print("\nPrincipal Components Ranked by Correlation with Stress:")
pc_corr_df_sorted
```

```
Principal Components Ranked by Correlation with Stress:
```

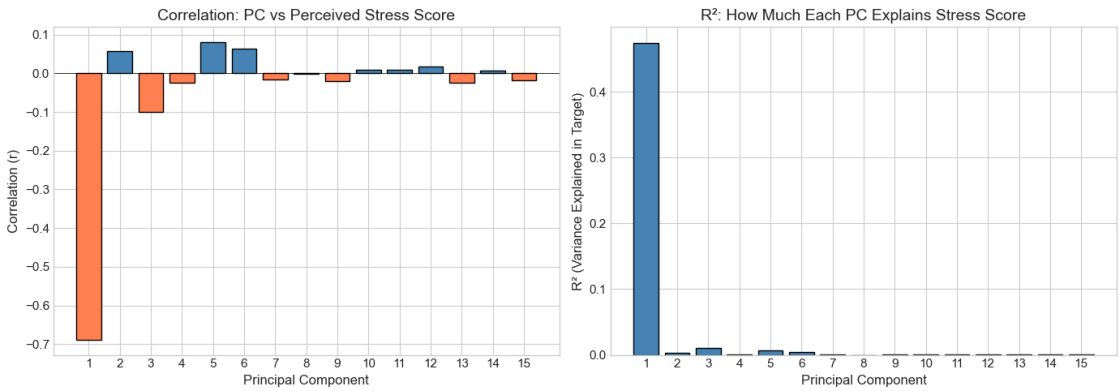
[53]:

	PC	r	R2	p_value	Abs_r
0	PC1	-0.689035	0.474770	0.000000e+00	0.689035
2	PC3	-0.099769	0.009954	0.000000e+00	0.099769
4	PC5	0.080974	0.006557	0.000000e+00	0.080974

```
5    PC6  0.064000  0.004096   0.000000e+00  0.064000
1    PC2  0.056989  0.003248   0.000000e+00  0.056989
12  PC13 -0.024402  0.000595  1.660388e-202  0.024402
3    PC4 -0.023661  0.000560  1.579817e-190  0.023661
8    PC9 -0.020727  0.000430  1.133205e-146  0.020727
14  PC15 -0.018468  0.000341  7.592765e-117  0.018468
11  PC12  0.017050  0.000291  7.006390e-100  0.017050
6    PC7 -0.016302  0.000266   1.795118e-91  0.016302
9   PC10  0.009387  0.000088   1.627287e-31  0.009387
10  PC11  0.008341  0.000070   3.153535e-25  0.008341
13  PC14  0.007565  0.000057   4.851992e-21  0.007565
7    PC8 -0.002265  0.000005   4.825443e-03  0.002265
```

[54]:

```
# Visualize PC-target correlations
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Correlation coefficients
colors = ['steelblue' if r >= 0 else 'coral' for r in pc_corr_df['r']]
axes[0].bar(range(1, len(eigenvalues)+1), pc_corr_df['r'], color=colors,␣
 ↪edgecolor='black')
axes[0].set_xlabel('Principal Component', fontsize=12)
axes[0].set_ylabel('Correlation (r)', fontsize=12)
axes[0].set_title('Correlation: PC vs Perceived Stress Score', fontsize=14)
axes[0].axhline(y=0, color='black', linewidth=0.5)
axes[0].set_xticks(range(1, len(eigenvalues)+1))

# Plot 2: R² values (variance explained in target)
axes[1].bar(range(1, len(eigenvalues)+1), pc_corr_df['R2'], color='steelblue',␣
 ↪edgecolor='black')
axes[1].set_xlabel('Principal Component', fontsize=12)
axes[1].set_ylabel('R² (Variance Explained in Target)', fontsize=12)
axes[1].set_title('R²: How Much Each PC Explains Stress Score', fontsize=14)
axes[1].set_xticks(range(1, len(eigenvalues)+1))

plt.tight_layout()
plt.savefig('pc_target_correlation.png', dpi=150, bbox_inches='tight')
plt.show()
```



1.12 11. Linear Regression Using Principal Components

Perform regression using principal components to predict the target variable.

[55]:
```
# Regression using all PCs
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Split data
X_pc_train, X_pc_test, y_train, y_test = train_test_split(
    X_pc, y, test_size=0.2, random_state=42
)

# Fit linear regression on PC space
lr_pc = LinearRegression()
lr_pc.fit(X_pc_train, y_train)

# Predict
y_pred = lr_pc.predict(X_pc_test)

# Evaluate
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Linear Regression in Principal Component Space:")
print("=" * 50)
print(f"  R² Score: {r2:.4f}")
print(f"  RMSE: {rmse:.4f}")
print(f"\nThis R² represents the linear relationship between")
print(f"all {len(eigenvalues)} principal components and perceived_stress_score.
↪")
```

```
Linear Regression in Principal Component Space:
==================================================
  R² Score: 0.5004
  RMSE: 8.3716

This R² represents the linear relationship between
all 15 principal components and perceived_stress_score.
```

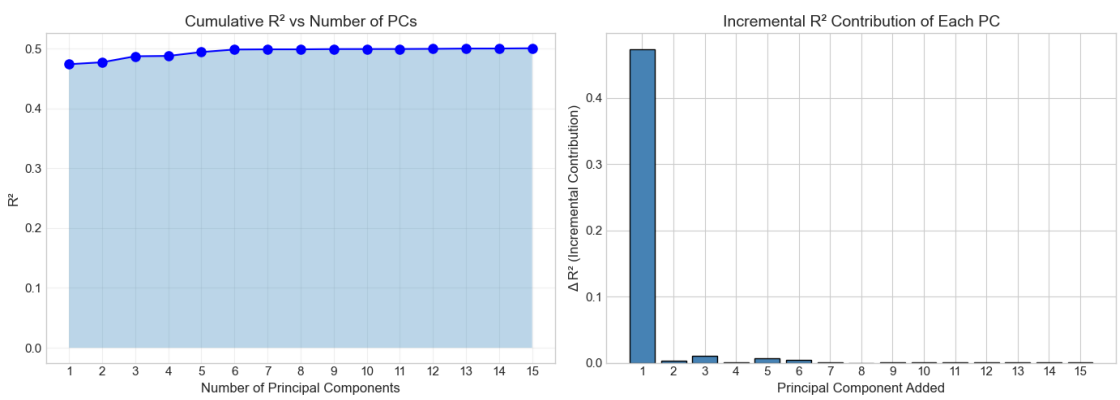
[56]:
```python
# Analyze regression coefficients in PC space
print("\nRegression Coefficients in PC Space:")
print("=" * 50)
print(f"{'PC':<8} {'Coefficient':<15} {'Eigenvalue':<15}")
print("-" * 50)

for i, (coef, ev) in enumerate(zip(lr_pc.coef_, eigenvalues)):
    print(f"PC{i+1:<6} {coef:<15.4f} {ev:<15.4f}")
```

```
Regression Coefficients in PC Space:
==================================================
PC       Coefficient     Eigenvalue
--------------------------------------------------
PC1      -3.4279         5.6599
PC2      0.5276          1.6330
PC3      -1.0205         1.3346
PC4      -0.2897         0.9445
PC5      1.0510          0.8308
PC6      0.9133          0.6831
PC7      -0.2416         0.6094
PC8      -0.0383         0.5912
PC9      -0.3279         0.5548
PC10     0.1607          0.5134
PC11     0.1498          0.4329
PC12     0.3149          0.4224
PC13     -0.4946         0.3562
PC14     0.1537          0.3441
PC15     -0.7107         0.0896
```

[57]:
```python
# Incremental R² analysis: How much does each PC add?
print("\nIncremental R² Analysis:")
print("=" * 60)
print(f"{'# PCs':<10} {'R²':<15} {'RMSE':<15} {'Δ R²':<15}")
print("-" * 60)

prev_r2 = 0
incremental_r2 = []

for n_pc in range(1, len(eigenvalues) + 1):
```

```
    # Use first n_pc components
    lr_temp = LinearRegression()
    lr_temp.fit(X_pc_train[:, :n_pc], y_train)
    y_pred_temp = lr_temp.predict(X_pc_test[:, :n_pc])

    r2_temp = r2_score(y_test, y_pred_temp)
    rmse_temp = np.sqrt(mean_squared_error(y_test, y_pred_temp))
    delta_r2 = r2_temp - prev_r2

    incremental_r2.append({'n_pc': n_pc, 'R2': r2_temp, 'RMSE': rmse_temp,␣
↪'Delta_R2': delta_r2})
    print(f"{n_pc:<10} {r2_temp:<15.4f} {rmse_temp:<15.4f} {delta_r2:<+15.4f}")

    prev_r2 = r2_temp
```

```
Incremental R² Analysis:
==============================================================
# PCs      R²              RMSE            Δ R²
--------------------------------------------------------------
1          0.4737          8.5918          +0.4737
2          0.4770          8.5653          +0.0032
3          0.4870          8.4826          +0.0101
4          0.4875          8.4782          +0.0005
5          0.4941          8.4240          +0.0065
6          0.4983          8.3891          +0.0042
7          0.4986          8.3865          +0.0003
8          0.4986          8.3865          +0.0000
9          0.4990          8.3828          +0.0004
10         0.4991          8.3823          +0.0001
11         0.4991          8.3817          +0.0001
12         0.4994          8.3795          +0.0003
13         0.4999          8.3754          +0.0005
14         0.4999          8.3750          +0.0001
15         0.5004          8.3716          +0.0004
```

[58]:
```
# Visualize incremental R²
inc_r2_df = pd.DataFrame(incremental_r2)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Cumulative R²
axes[0].plot(inc_r2_df['n_pc'], inc_r2_df['R2'], 'bo-', markersize=8)
axes[0].fill_between(inc_r2_df['n_pc'], 0, inc_r2_df['R2'], alpha=0.3)
axes[0].set_xlabel('Number of Principal Components', fontsize=12)
axes[0].set_ylabel('R²', fontsize=12)
axes[0].set_title('Cumulative R² vs Number of PCs', fontsize=14)
```

```
axes[0].set_xticks(inc_r2_df['n_pc'])
axes[0].grid(True, alpha=0.3)

# Plot 2: Incremental R² (contribution of each PC)
colors = ['steelblue' if d >= 0 else 'coral' for d in inc_r2_df['Delta_R2']]
axes[1].bar(inc_r2_df['n_pc'], inc_r2_df['Delta_R2'], color=colors,␣
 ↪edgecolor='black')
axes[1].set_xlabel('Principal Component Added', fontsize=12)
axes[1].set_ylabel('Δ R² (Incremental Contribution)', fontsize=12)
axes[1].set_title('Incremental R² Contribution of Each PC', fontsize=14)
axes[1].set_xticks(inc_r2_df['n_pc'])
axes[1].axhline(y=0, color='black', linewidth=0.5)

plt.tight_layout()
plt.savefig('incremental_r2_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```